

ESPECIFICACIÓN DE REQUISITOS DE SOFTWARE

RENTAMOVIL

**Thiago Fabian Rojas Guevara — David Santiago Erez Rodríguez — Nicole Dayana
Ramírez Vargas**

ANÁLISIS Y DESARROLLO DE SOFTWARE 3145556

SERVICIO NACIONAL DE APRENDIZAJE

2026

1. INTRODUCCIÓN

El proyecto RentaMovil consiste en el desarrollo de una plataforma web y móvil para optimizar el alquiler de vehículos particulares. Su objetivo es agilizar el proceso de reservas, reducir tiempos de espera y mejorar el control, seguimiento y seguridad de las flotas mediante herramientas de geolocalización, notificaciones y gestión en línea. Este documento SRS define el alcance, los objetivos y los requisitos funcionales y no funcionales del sistema, sirviendo como guía para garantizar que el producto final cumpla con las necesidades de los usuarios y de la empresa.

1.1 Título: RentaMovil

Renta y control de automóviles particulares.

1.2 Problema o necesidad

Se presentan problemáticas relacionadas con la congestión en el proceso de reservas y los extensos tiempos de espera para el alquiler de vehículos en los puntos físicos. Además, se presenta una deficiente gestión en el control, seguimiento y seguridad de las flotas vehiculares alquiladas.

¿Cómo puede implementarse una solución tecnológica que optimice el proceso de reservas, reduzca los tiempos de espera y mejore el control, seguimiento y seguridad de las flotas vehiculares alquiladas?

1.3 Justificación

El crecimiento del mercado de alquiler de vehículos ha evidenciado deficiencias en la gestión tradicional, especialmente en los puntos físicos, donde las reservas manuales y los largos tiempos de espera afectan la experiencia del usuario. Según Statista (2024), esta industria alcanzará los \$120 mil millones en 2026, lo que exige mayor eficiencia operativa. Estudios de McKinsey (2023) muestran que la digitalización puede reducir los tiempos de espera hasta en un 60% y mejorar la eficiencia en un 35%. Además, la International Transport Forum destaca que la tecnología mejora el control y seguridad de las flotas. Por ello, es fundamental implementar una solución tecnológica integral que optimice las reservas y fortalezca el seguimiento de vehículos.

1.4 Objetivo general

Diseñar e implementar un sistema tecnológico integral que optimice el proceso de reserva de vehículos y fortalezca el control, seguimiento y seguridad de las flotas vehiculares alquiladas.

1.5 Objetivos específicos

1. Analizar las principales deficiencias en el proceso actual de reservas y gestión de flotas en empresas de alquiler de vehículos.
2. Diseñar una plataforma tecnológica que permita realizar reservas en línea de forma eficiente y en tiempo real.
3. Implementar herramientas de geolocalización y seguimiento para mejorar el control y la seguridad de los vehículos alquilados, limitado a vehículos actualmente en alquiler.
4. Agregar funcionalidades administrativas para la completa gestión de las respectivas flotas de vehículos, incluyendo su ciclo de mantenimiento.
5. Integrar funcionalidades de notificación in-app para eventos relevantes de reservas y pagos.

6. Evaluar el impacto del sistema propuesto en la reducción de tiempos de espera y mejora de la seguridad operativa.

1.6 Alcance

El presente proyecto se enfoca en el diseño, desarrollo e implementación de una solución tecnológica integral para la gestión de reservas y control de flotas vehiculares en empresas de alquiler de automóviles. El sistema estará dirigido tanto a clientes como a administradores del servicio.

Incluye:

- Digitalización del proceso de reservas mediante una plataforma web y móvil.
- Registro e inicio de sesión de clientes, con recuperación de contraseña por código temporal.
- Gestión del perfil del cliente: nombre, teléfono (opcional), foto de perfil y cambio de contraseña desde la sesión activa; el cambio de email requiere confirmar con la contraseña actual.
- Selección opcional de seguro y aceptación de términos y condiciones durante la reserva.
- Pago por transferencia bancaria: el cliente escanea un código QR de una cuenta bancaria de la empresa, transfiere y sube el comprobante; un Administrador lo revisa y aprueba o rechaza (no hay integración con una pasarela de pago automatizada — la verificación es manual).
- Ejecución del alquiler (recogida/devolución) con generación de factura.
- Geolocalización para seguimiento en tiempo real, limitada a vehículos en estado alquilado.
- Módulo de estado y disponibilidad de la flota, con inventario filtrable por estado para administración.
- Gestión del ciclo de vida de mantenimiento de vehículos (programado, en curso, completado o cancelado), con historial por vehículo.
- Gestión de sucursales, incluida su ubicación en mapa (latitud/longitud).
- Notificaciones in-app para clientes y administradores.

No incluye:

- Validación de licencia de conducción.
- Integración con una pasarela de pago automatizada (procesador de tarjetas/PSE); la verificación del pago sigue siendo manual, por un Administrador, no una llamada a un proveedor externo.
- Aplicación móvil para administradores (solo web).
- Módulo completo de gestión de órdenes de mantenimiento con talleres externos o cotizaciones — el sistema registra el ciclo de vida del mantenimiento (programado/en curso/completado/cancelado), costo y observaciones, pero no un flujo de cotización con terceros.
- Concepto de "conductor" distinto al cliente/titular de la reserva.

1.7 Personal Involucrado

Nombre	Rol	Categoría Profesional	Responsabilidad	Contacto
Nicole Dayana Ramírez Vargas	Líder	Aprendiz del tecnólogo en análisis y desarrollo de software	Liderar, planificar y supervisar la entrega del proyecto	nicolerv18007@gmail.com
David Santiago Erez Rodríguez	Desarrollador	Aprendiz del tecnólogo en	Programar el aplicativo web	erezsantiago25@gmail.com

Nombre	Rol	Categoría Profesional	Responsabilidad	Contacto
		análisis y desarrollo de software		
Thiago Fabian Rojas Guevara	Analista	Aprendiz del tecnólogo en análisis y desarrollo de software	Análisis de información, diseño y programación	thiagorojasg451@gmail.com

1.8 Definiciones, acrónimos y abreviaturas

Término	Descripción
Usuario	Persona que usará la aplicación (Cliente, Administrador o Super Admin)
SRS	Especificación de requisitos del software
RF	Requisito funcional
RNF	Requisito no funcional
SENA	Servicio Nacional de Aprendizaje
HU	Historia de Usuario, el formato en el que se levantó cada requisito antes de redactarlo como RF

1.9 Referencias

Título del documento	Referencia
Standard IEEE 830-1998	IEEE
Documentación técnica interna del equipo (historias de usuario, modelo de dominio, contratos de API)	repositorio <code>rtm-docs</code>

1.10 Resumen

El proyecto RentaMovil surge como una respuesta a las deficiencias actuales en el sector de alquiler de vehículos, específicamente en los procesos manuales de reserva y en el control de las flotas. A través de este sistema se permitirá realizar reservas en línea, rastrear en tiempo real los vehículos durante el alquiler y facturar automáticamente cada reserva pagada. Con un enfoque centrado en la eficiencia, la seguridad y la experiencia del cliente, RentaMovil se proyecta como una herramienta clave para modernizar la gestión del alquiler de vehículos.

2. DESCRIPCIÓN GENERAL

2.1 Perspectiva del Producto

RentaMovil es una solución tecnológica integral diseñada para digitalizar y optimizar el proceso de alquiler de vehículos particulares. El backend se implementa como **microservicios en Java + Spring Boot con Arquitectura Hexagonal**, organizados en 7 Bounded Contexts (Identity & Access, Fleet & Maintenance, Booking & Reservation, Rental Execution, Payment & Billing, Telemetry & GPS, Notification), más **audit** como infraestructura transversal que los 7 escriben directamente, sobre una **base de datos SQL Server compartida**, migrada con **Liquibase**. El frontend se entrega como una aplicación **web (React + Vite)** y una aplicación **móvil (Expo + React Native)**, ambas para clientes; la gestión administrativa se realiza exclusivamente vía web. El almacenamiento de archivos (comprobantes de pago, códigos QR, fotos de perfil, fotos de vehículos y de mantenimiento) se realiza en **Cloudinary**.

2.2 Características de los usuarios

Rol	Formación	Actividades
Cliente (CLIENT)	Natural	Explora el catálogo, selecciona vehículo y seguro, crea y gestiona sus reservas, paga, consulta sus facturas, administra su perfil
Administrador (ADMIN)	Administración de empresas o afin	Gestiona flota, sucursales, mantenimiento, reservas, pagos y facturación de toda la empresa; registra recogidas/devoluciones; consulta ubicación GPS
Super Admin (SUPER_ADMIN)	Administracion de empresas o afin	Todo lo del Administrador, más la gestión de usuarios administrativos (otorgar/revocar roles) y de cuentas bancarias

Nota de diseño: un Administrador o Super Admin también puede ser titular de una reserva personal (usando su propia cuenta como cliente) — el rol determina qué puede administrar, no quién puede ser titular de una reserva.

2.3 Restricciones

- 2.3.1. La aplicación requiere conexión a internet para su funcionamiento.
- 2.3.2. Los usuarios deben contar con la última versión de la aplicación móvil para garantizar compatibilidad y seguridad.
- 2.3.3. El software debe ser compatible con dispositivos móviles (Android/iOS) y navegadores web modernos.
- 2.3.4. Se deben cumplir las normativas de protección de datos personales, ya que se maneja información sensible de usuarios y vehículos.
- 2.3.5. La precisión del sistema de geolocalización depende de la calidad del GPS del dispositivo y la red disponible.
- 2.3.6. El backend debe implementarse en Java + Spring Boot con Arquitectura Hexagonal, sobre una base de datos SQL Server compartida entre todos los microservicios, migrada con Liquibase.

- 2.3.7. El pago se procesa por **transferencia bancaria con revisión manual**: el cliente sube un comprobante y un Administrador lo aprueba o rechaza — no hay integración con una pasarela de pago automatizada.
- 2.3.8. El rastreo GPS solo aplica a vehículos en estado **RENTED** (actualmente alquilados).

2.4 Suposiciones y Dependencias

- 2.4.1. Se asume que los requisitos descritos son estables y aceptados por las partes interesadas.
 - 2.4.2. Los dispositivos donde se ejecute la aplicación deben cumplir con los requisitos técnicos establecidos.
 - 2.4.3. Las reservas y notificaciones se gestionan en tiempo real, siempre que haya conexión estable.
 - 2.4.4. El éxito del sistema dependerá de la adopción y correcta interacción por parte de los usuarios (clientes y administradores).
 - 2.4.5. El sistema podrá escalar y permitir el registro de nuevas unidades vehiculares conforme crezca la demanda.
 - 2.4.6. Cada usuario tiene exactamente un rol (**CLIENT**, **ADMIN** o **SUPER_ADMIN**) — no hay usuarios multi-rol.
-

3. REQUISITOS ESPECÍFICOS

3.1 Requisitos comunes de las interfaces

3.1.1 Interfaz de Usuario (Cliente)

- Visualización y filtrado de vehículos disponibles.
- Selección de vehículo y seguro opcional.
- Creación y gestión de reservas propias.
- Pago de reservas por transferencia bancaria (subida de comprobante).
- Consulta y descarga de facturas.
- Gestión de perfil (nombre, teléfono, foto, email, contraseña).
- Recepción de notificaciones in-app.

3.1.2 Interfaz de Usuario (Administrador / Super Admin)

- Gestión de vehículos (registro, edición, baja).
- Gestión de sucursales, incluida su ubicación en mapa.
- Gestión del ciclo de mantenimiento de la flota (programar, iniciar, completar, cancelar).
- Administración de reservas de toda la empresa.
- Registro de recogida/devolución de vehículos.
- Consulta de pagos y facturación.
- Rastreo GPS y consulta de historial de rutas.
- Gestión de usuarios administrativos y de cuentas bancarias (exclusivo de Super Admin).

3.1.3 Interfaces de hardware

- Conexión estable a internet.
- Versión reciente del software y sistema operativo.
- Al menos 500 MB de almacenamiento libre para la aplicación móvil.

3.1.4 Interfaces de software

- **Sistema Operativo (PC):** Windows 7 o superior, macOS 11 o superior, distribuciones Linux modernas.
- **Sistema Operativo (Móviles):** Android 10 o superior, iOS/iPadOS 15 o superior.
- **Explorador:** Chrome, Firefox, Edge, Safari y navegadores modernos equivalentes.
- **Backend:** Java + Spring Boot, SQL Server, Liquibase.

3.1.5 Interfaces de comunicación

La comunicación entre clientes, servidores y servicios externos se realiza mediante protocolos seguros HTTPS y TLS 1.2+. La comunicación interna entre microservicios usa mTLS o API key interna, según el servicio.

3.2 Requisitos funcionales — resumen

ID	Módulo (Bounded Context)	Descripción	Servicio responsable
RF1	Identity & Access	Registro, autenticación y gestión de cuenta del cliente	identity-access-service
RF2	Fleet & Maintenance	Catálogo público de vehículos y consulta de disponibilidad	fleet-maintenance-service
RF3	Booking & Reservation	Creación, consulta, modificación y cancelación de reservas propias	booking-reservation-service
RF4	Rental Execution	Registro de recogida y devolución física del vehículo	rental-execution-service
RF5	Payment & Billing	Pago por transferencia con revisión de comprobante, confirmación de reserva, facturación propia	payment-billing-service
RF6	Telemetry & GPS	Rastreo de ubicación y consulta de historial de rutas	telemetry-gps-service
RF7	Notification	Notificaciones in-app	notification-service
RF8	Administración (transversal)	Gestión administrativa de usuarios, flota, mantenimiento, sucursales, reservas, seguros y pagos — ver campo "Dominio" en cada ítem	varios (ver dominio por ítem)

3.3 Descripción de Requisitos Funcionales

RF 1.1 — Registro de cliente

Campo	Valor
Identificador	RF 1.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Documentos de visualización asociados: formulario de registro (nombre, apellido, email, username, teléfono opcional, contraseña), mensajes de validación y confirmación.

Entrada: el visitante completa nombre, apellido, email, username y contraseña, y opcionalmente un teléfono, y envía el formulario.

Salida: se crea la Person y el User (rol **CLIENT**); confirmación de cuenta creada.

Descripción: el sistema permite a un visitante no autenticado registrarse para poder crear reservas. El username es elegido por el visitante y debe ser único; el teléfono es opcional — si no se envía, la persona queda sin teléfono registrado.

Manejo de situaciones anormales: si el email ya está registrado, el sistema rechaza el registro con el mensaje correspondiente; si el username ya está tomado, el sistema lo rechaza con un mensaje distinto, sin crear una cuenta duplicada.

Criterios de aceptación:

- El formulario exige nombre, apellido, email, username y contraseña; el teléfono es opcional.
- El email y el username deben ser únicos en el sistema.
- La cuenta creada recibe automáticamente el rol **CLIENT**.

RF 1.2 — Inicio de sesión

Campo	Valor
Identificador	RF 1.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Documentos de visualización asociados: formulario de login (email/username + contraseña).

Entrada: el usuario ingresa sus credenciales.

Salida: access token (30 min) y refresh token (7 días) si las credenciales son válidas.

Descripción: el sistema autentica al usuario y emite los tokens JWT correspondientes.

Manejo de situaciones anormales: si las credenciales son inválidas, el sistema responde 401 sin indicar si el email existe o no (para no filtrar información) y la cuenta de la persona nunca se ve afectada. Para frenar fuerza bruta, el sistema limita los intentos por IP de origen: tras 10 intentos fallidos desde la misma IP (contra cualquier cuenta), esa IP queda bloqueada para intentar login durante 5 minutos (responde 429). El contador es por IP, no por cuenta — así un atacante no puede bloquear la cuenta de otra persona a propósito.

Criterios de aceptación:

- Solo usuarios con `status = ACTIVE` pueden iniciar sesión.
- El access token expira a los 30 minutos; el refresh token a los 7 días, con rotación en cada uso (un refresh token se usa una sola vez).
- Al décimo intento fallido desde una misma IP, esa IP queda bloqueada 5 minutos para el endpoint de login (responde 429); pasado ese tiempo puede volver a intentar.
- Ninguna cuenta se bloquea automáticamente por intentos fallidos de login.

RF 1.3 — Recuperación de contraseña

Campo	Valor
Identificador	RF 1.3
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Documentos de visualización asociados: formulario de solicitud de recuperación, formulario de nueva contraseña.

Entrada: el usuario solicita recuperación con su email; recibe un código de 6 dígitos válido por 15 minutos y lo ingresa junto con la contraseña nueva.

Salida: contraseña actualizada; el código queda marcado como usado; se cierran todas las sesiones activas del usuario.

Descripción: el sistema permite recuperar el acceso mediante un código temporal enviado al email registrado. La solicitud siempre responde de forma genérica (202), exista o no el email, para no revelar qué correos están registrados en el sistema.

Manejo de situaciones anormales: si el código está expirado, ya fue usado, o no corresponde al último emitido para esa cuenta, el sistema rechaza la operación sin actualizar la contraseña.

Criterios de aceptación:

- El código tiene 6 dígitos y vence a los 15 minutos; solo el último código emitido es válido.
- Un código usado no puede reutilizarse.
- Al completar la recuperación, todas las sesiones activas del usuario se cierran.

RF 1.4 — Gestión de información de cuenta

Campo	Valor
Identificador	RF 1.4
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Media

Entrada: el cliente autenticado actualiza su nombre, teléfono y/o foto de perfil desde su perfil; o, para cambiar su email, ingresa el nuevo email junto con su contraseña actual.

Salida: datos de la Person actualizados; si fue un cambio de email, el nuevo valor queda reflejado en el perfil solo tras confirmar la contraseña.

Descripción: el sistema permite mantener actualizada la información personal del cliente. La foto de perfil es una URL ya subida a Cloudinary desde el frontend; el sistema solo acepta URLs de la cuenta de Cloudinary del proyecto. El username no puede modificarse una vez creada la cuenta.

Manejo de situaciones anormales: si la contraseña ingresada para cambiar el email no coincide, el sistema rechaza el cambio y el email permanece igual; si el nuevo email ya está registrado por otra Person, se rechaza con el mensaje correspondiente; una foto cuya URL no pertenece a la cuenta de Cloudinary del proyecto se rechaza.

Criterios de aceptación:

- Nombre, teléfono y foto se actualizan sin confirmación adicional.
- El cambio de email exige reingresar la contraseña actual y solo se aplica si coincide.
- El username permanece inmutable desde esta pantalla.

RF 1.5 — Cambio de contraseña (sesión activa)

Campo	Valor
Identificador	RF 1.5
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Media

Entrada: el cliente autenticado ingresa su contraseña actual y la nueva contraseña.

Salida: contraseña actualizada; se cierran todas las sesiones del usuario, incluida la actual.

Descripción: permite a un usuario ya autenticado cambiar su contraseña desde su perfil, sin pasar por el flujo de recuperación por código (RF1.3), reutilizando la contraseña actual como confirmación de identidad.

Manejo de situaciones anormales: si la contraseña actual no coincide, el sistema la rechaza; si la nueva contraseña es igual a la actual, también se rechaza.

Criterios de aceptación:

- La nueva contraseña debe cumplir la misma regla que el registro (mínimo 8 caracteres, al menos una mayúscula y un número).
- Tras el cambio, el usuario debe iniciar sesión de nuevo en todos sus dispositivos.

RF 2.1 — Explorar y filtrar catálogo de vehículos

Campo	Valor
Identificador	RF 2.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: un visitante (con o sin sesión) accede al catálogo y opcionalmente aplica filtros (categoría, marca, motor, precio, fechas).

Salida: listado de vehículos con marca, modelo, categoría, precio diario e imagen; o estado vacío si no hay coincidencias.

Descripción: el sistema permite explorar el catálogo público de vehículos sin necesidad de autenticarse. Sin un filtro de estado explícito, el catálogo solo muestra vehículos **AVAILABLE**. El mismo endpoint, con sesión de Administrador, admite un filtro de estado adicional para uso administrativo (ver RF8.2).

Manejo de situaciones anormales: si no hay vehículos en estado **AVAILABLE**, o el filtro no arroja resultados, se muestra un estado vacío sin error. Combinar el filtro de fechas con un estado distinto de **AVAILABLE** se rechaza, porque el filtro de fechas solo tiene sentido sobre el catálogo público.

Criterios de aceptación:

- El catálogo es accesible sin sesión iniciada.
- Los filtros se pueden combinar (categoría + marca + motor + precio).

RF 2.2 — Consultar disponibilidad de un vehículo

Campo	Valor
Identificador	RF 2.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el cliente consulta disponibilidad de un vehículo para un rango de fechas.

Salida: confirmación de disponibilidad o no, según el estado del vehículo y las reservas existentes que se crucen con el rango.

Descripción: el sistema calcula disponibilidad combinando `Vehicle.status` y las reservas activas del vehículo — no es un campo almacenado, se calcula en el momento.

Criterios de aceptación:

- Un vehículo con una reserva que se cruza con el rango pedido se marca como no disponible.
- Un vehículo `AVAILABLE` sin cruces se marca como disponible.

RF 3.1 — Crear reserva (selección de vehículo, seguro, fechas y sucursales)

Campo	Valor
Identificador	RF 3.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el cliente autenticado selecciona un vehículo disponible, opcionalmente un plan de seguro (First Aid, Standard o All-Risk), fechas de recogida/devolución, sucursal de recogida y de devolución (pueden diferir sin costo extra), y acepta los términos y condiciones.

Salida: `Reservation` creada en estado `PENDING_PAYMENT`; evento `ReservationCreated`.

Descripción: el sistema crea la reserva calculando `vehicle_subtotal`, `insurance_subtotal` (si aplica) y `total_amount`. El término "conductor" no existe en este modelo — el titular de la reserva (`client_id`) es siempre la Person autenticada que la crea.

Manejo de situaciones anormales: si `end_date ≤ start_date`, o el cliente no aceptó los términos y condiciones, la reserva se rechaza.

Criterios de aceptación:

- Una reserva tiene exactamente un vehículo, no modificable después de creada.
- Como máximo un seguro por reserva, cobrado una sola vez.
- Sucursal de recogida y devolución pueden diferir, sin cargo adicional.
- No se puede continuar sin aceptar los términos y condiciones.

RF 3.2 — Ver y modificar reservas propias

Campo	Valor
Identificador	RF 3.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el cliente consulta sus reservas y, si aplica, extiende la fecha de devolución, cambia la sucursal de devolución, o actualiza su teléfono en una reserva.

Salida: listado de reservas propias con su estado; o reserva actualizada con evento `ReservationReturnDateExtended` o `ReservationReturnBranchChanged` según el cambio.

Manejo de situaciones anormales: un intento de acortar `end_date` (en vez de extenderlo) se rechaza; un cambio de sucursal de devolución con menos de 3 días para `end_date` se rechaza.

Criterios de aceptación:

- Un cliente puede tener múltiples reservas activas simultáneamente.
- `end_date` solo puede extenderse, nunca acortarse.
- La sucursal de devolución solo puede cambiarse hasta 3 días antes de `end_date` — igual que el plazo de cancelación (RF3.3).

RF 3.3 — Cancelar reserva

Campo	Valor
Identificador	RF 3.3
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el cliente cancela una reserva propia.

Salida: `Reservation.status = CANCELLED`, sin cargo; evento `ReservationCancelled`.

Manejo de situaciones anormales: si faltan menos de 3 días para `start_date`, la cancelación se rechaza.

Criterios de aceptación:

- Cancelación permitida solo con ≥ 3 días de anticipación.
- No se genera ningún cargo por cancelar.

RF 4.1 — Registrar recogida del vehículo

Campo	Valor
Identificador	RF 4.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el Administrador, en sucursal, registra la recogida de un vehículo para una reserva `CONFIRMED`, con el kilometraje inicial y el GPS asignado.

Salida: `Rental` creado en `IN_PROGRESS`; `Vehicle.status = RENTED`; evento `RentalStarted`.

Manejo de situaciones anormales: una reserva que no está `CONFIRMED` no puede iniciar su `Rental`.

Criterios de aceptación:

- El `Rental` solo se crea desde una `Reservation CONFIRMED`.

RF 4.2 — Registrar devolución del vehículo

Campo	Valor
Identificador	RF 4.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el Administrador registra la devolución con el kilometraje final.

Salida: `Rental.status = COMPLETED`; `Vehicle.status = AVAILABLE`; `Reservation.status = COMPLETED`; evento `RentalCompleted`.

Manejo de situaciones anormales: un `final_mileage` menor al `initial_mileage` se rechaza.

Criterios de aceptación:

- `final_mileage` siempre debe ser $\geq$ `initial_mileage`.

RF 5.1 — Pagar una reserva por transferencia bancaria

Campo	Valor
Identificador	RF 5.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el cliente selecciona una `BankAccount` (cuenta bancaria de la empresa, mostrada como código QR), transfiere el `total_amount` exacto de su reserva por fuera del sistema, y sube el comprobante de pago. Un Administrador revisa después ese comprobante y lo aprueba o rechaza.

Salida: `Payment` creado en `PENDING_REVIEW`; `Reservation.status = PENDING_REVIEW`; evento `PaymentReceiptUploaded`. Tras la revisión del Administrador: si aprueba, `Payment.status = APPROVED`, `Reservation.status = CONFIRMED`, eventos `PaymentCompleted/ReservationConfirmed`; si rechaza, `Payment.status = REJECTED`, evento

`PaymentRejected`, y `Reservation.status` vuelve a `PENDING_PAYMENT` (si el Administrador permite reintentar) o pasa a `CANCELLED` (si el Administrador cancela la reserva).

Descripción: el pago es una transferencia bancaria real, verificada manualmente por un Administrador — no hay integración con una pasarela de pago automatizada (procesador de tarjetas/PSE) en este MVP. El comprobante y el código QR se almacenan en Cloudinary.

Manejo de situaciones anormales: si el monto no coincide exactamente con `total_amount`, el sistema rechaza la subida del comprobante antes de crear el `Payment` — no existen pagos parciales ni "saldo pendiente". Si no hay un comprobante aprobado dentro de las 24 horas siguientes a `reservation_date`, la reserva expira automáticamente a `CANCELLED`.

Criterios de aceptación:

- El monto declarado por el cliente debe ser exactamente igual al total de la reserva.
- Un comprobante rechazado no confirma la reserva; el Administrador decide si vuelve a `PENDING_PAYMENT` (reintento) o cancela la reserva.
- Sin comprobante aprobado en 24 horas, la reserva se cancela automáticamente.

RF 5.2 — Generar y consultar factura

Campo	Valor
Identificador	RF 5.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: tras un `PaymentCompleted`, el sistema genera la factura automáticamente; el cliente la consulta y descarga después.

Salida: `Invoice` con `invoice_number` consecutivo y PDF generado una sola vez y almacenado; evento `InvoiceGenerated`.

Descripción: el documento generado es una factura de facturación, no un contrato legal firmado digitalmente.

Manejo de situaciones anormales: no se genera una segunda factura para la misma reserva aunque el evento se reprocese (idempotencia por `reservation_id`).

Criterios de aceptación:

- La factura solo se genera si hubo un pago exitoso.
- El PDF servido en cada descarga es siempre el mismo archivo (no se regenera).

RF 6.1 — Rastrear ubicación del vehículo en alquiler

Campo	Valor
Identificador	RF 6.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: el Administrador consulta la ubicación de un vehículo en alquiler.

Salida: última `Location` registrada (latitud/longitud).

Manejo de situaciones anormales: un cliente (`role = CLIENT`) que intente acceder a este endpoint recibe 403.

Criterios de aceptación:

- Solo se rastrea mientras `Vehicle.status = RENTED`.
- Solo ADMIN/SUPER_ADMIN pueden consultar ubicación.

RF 6.2 — Consultar historial de rutas

Campo	Valor
Identificador	RF 6.2
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Media

Entrada: el Administrador consulta la ruta completa de un alquiler ya finalizado.

Salida: conjunto de puntos `Location` en orden cronológico.

Manejo de situaciones anormales: si no hay puntos registrados (falla de GPS), se muestra vacío con una nota informativa, sin error.

Criterios de aceptación:

- La ruta corresponde exactamente al período `IN_PROGRESS` del Rental consultado.

RF 7.1 — Notificaciones in-app

Campo	Valor
Identificador	RF 7.1
Tipo	Requisito Funcional
¿Crítico?	Sí
Prioridad	Alta

Entrada: eventos de dominio (`ReservationCreated`, `PaymentReceiptUploaded`, `PaymentCompleted`, `PaymentRejected`, `ReservationCancelled`, `ReservationExpired`, etc.).

Salida: `Notification` creada para el destinatario correspondiente (cliente o Administrador).

Descripción: el sistema notifica dentro de la aplicación sobre eventos relevantes de reservas y pagos, sin depender de correo/SMS en este MVP.

Criterios de aceptación:

- Cada evento relevante genera una notificación al destinatario correcto (cliente vs. Administrador).
- El destinatario puede marcar una notificación como leída.

RF 8.1 — Gestión administrativa de usuarios

Campo	Valor
Identificador	RF 8.1
Tipo	Requisito Funcional
Dominio	Identity & Access
¿Crítico?	Sí
Prioridad	Alta

Entrada: un Super Admin selecciona una Person existente y le asigna rol `ADMIN` o `SUPER_ADMIN`; también puede desactivar una cuenta `ACTIVE` o reactivarla.

Salida: `user.role_id` o `user.status` actualizado; se emite el evento `RoleGranted` cuando aplica.

Descripción: el sistema permite al Super Admin delegar o revocar acceso administrativo, incluyendo a otros Super Admin, y controlar el estado de cualquier cuenta.

Manejo de situaciones anormales: si un usuario con rol `ADMIN` (no Super Admin) intenta cambiar roles o estados, el sistema responde 403 — esta operación es exclusiva de `SUPER_ADMIN`.

Criterios de aceptación:

- Solo `SUPER_ADMIN` tiene el permiso de otorgar/revocar roles y cambiar el estado de una cuenta.
- Desactivar una cuenta (`INACTIVE`) no borra su historial de reservas ni pagos.
- El cambio de rol o de estado queda auditado.

RF 8.2 — Administrar vehículos (registrar, editar, dar de baja, inventario)

Campo	Valor
Identificador	RF 8.2
Tipo	Requisito Funcional

Campo	Valor
Dominio	Fleet & Maintenance
¿Crítico?	Sí
Prioridad	Alta

Entrada: el Administrador registra un vehículo nuevo (placa, modelo, categoría, motor, precio diario, sucursal) o edita/elimina uno existente; también puede listar el inventario completo filtrando por estado (AVAILABLE, RENTED, MAINTENANCE, RETIRED o ALL).

Salida: vehículo creado en estado AVAILABLE; o actualizado/eliminado; evento VehicleRegistered si aplica; o listado de inventario filtrado.

Descripción: el sistema permite al Administrador mantener actualizado el catálogo de la flota y consultarlo como inventario interno, a diferencia del catálogo público de RF2.1 que solo expone vehículos AVAILABLE.

Manejo de situaciones anormales: una placa duplicada se rechaza; un vehículo con una reserva futura no cancelada, o en estado RENTED, no puede eliminarse hasta que no tenga reservas pendientes y vuelva a AVAILABLE.

Criterios de aceptación:

- La placa es única en el sistema.
- No se puede eliminar un vehículo con reserva activa.
- El filtro de inventario por estado requiere sesión de Administrador; sin autenticación solo se obtiene el catálogo público AVAILABLE.

RF 8.3 — Enviar y completar mantenimiento de un vehículo

Campo	Valor
Identificador	RF 8.3
Tipo	Requisito Funcional
Dominio	Fleet & Maintenance
¿Crítico?	Sí
Prioridad	Alta

Entrada: el Administrador marca un vehículo AVAILABLE para mantenimiento; más adelante, marca ese mantenimiento como finalizado.

Salida: al enviarlo, Vehicle.status = MAINTENANCE y se crea un VehicleMaintenance en IN_PROGRESS, con evento VehicleSentToMaintenance; al completarlo, el VehicleMaintenance pasa a COMPLETED con end_date, Vehicle.status vuelve a AVAILABLE, y se emite MaintenanceCompleted.

Descripción: el sistema permite controlar el estado operativo de cada vehículo y ver estadísticas agregadas por estado. El detalle de programación, edición, cancelación e historial del mantenimiento se describe en RF8.7.

Manejo de situaciones anormales: un vehículo **RENTED** no puede enviarse a mantenimiento directamente.

Criterios de aceptación:

- Los 4 estados válidos de **Vehicle** son **AVAILABLE**, **RENTED**, **MAINTENANCE**, **RETIRED**. **RETIRED** es baja lógica — el vehículo no se borra físicamente, solo deja de aparecer en el catálogo y no se puede volver a reservar.
- El panel de estadísticas muestra el conteo de vehículos por estado.

RF 8.4 — Gestión de sucursales

Campo	Valor
Identificador	RF 8.4
Tipo	Requisito Funcional
Dominio	Fleet & Maintenance
¿Crítico?	Sí
Prioridad	Media

Entrada: el Administrador crea, edita o elimina una sucursal, y puede ubicarla en un mapa.

Salida: sucursal disponible como opción de recogida/devolución, con su latitud/longitud almacenada si fue ubicada en el mapa.

Descripción: la ubicación en mapa es opcional — una sucursal creada sin coordenadas sigue siendo válida, solo que no se dibuja en los mapas de la aplicación.

Manejo de situaciones anormales: no se puede eliminar una sucursal con vehículos asignados; no se puede guardar una sucursal con solo una de las dos coordenadas (latitud o longitud) presente.

Criterios de aceptación:

- Una sucursal con vehículos asociados no puede eliminarse hasta reasignarlos.
- Latitud y longitud se guardan ambas o ninguna.

RF 8.5 — Administración de reservas y de seguros

Campo	Valor
Identificador	RF 8.5
Tipo	Requisito Funcional
Dominio	Booking & Reservation

Campo	Valor
¿Crítico?	Sí
Prioridad	Media

Entrada: el Administrador consulta todas las reservas del sistema (no solo las propias), y gestiona los planes de seguro disponibles (`InsuranceType`).

Salida: listado completo de reservas con filtros; al abrir el detalle de una reserva, se muestra también su información de pago (monto, cuenta bancaria usada, comprobante subido, y estado `PENDING_REVIEW/APPROVED/REJECTED`); planes de seguro creados/editados.

Descripción: no existe una pantalla separada de "consulta de pagos" — el pago se ve como parte del detalle de la reserva a la que pertenece, no como un módulo aparte.

Criterios de aceptación:

- El Administrador ve todas las reservas, incluidas las suyas propias como cliente, sin tratamiento especial.
- El detalle de cada reserva incluye su información de pago asociada.
- Un cambio en `daily_cost` de un seguro no afecta reservas ya creadas.

RF 8.6 — Gestión de cuentas bancarias

Campo	Valor
Identificador	RF 8.6
Tipo	Requisito Funcional
Dominio	Payment & Billing
¿Crítico?	Sí
Prioridad	Alta

Entrada: un Super Admin crea, edita o desactiva una `BankAccount` (banco, titular, imagen del código QR).

Salida: cuenta bancaria disponible (o no) como opción de pago en el checkout del cliente.

Descripción: exclusivo de `SUPER_ADMIN` — ni `CLIENT` ni `ADMIN` pueden gestionar las cuentas bancarias de la empresa, por ser información financiera sensible.

Manejo de situaciones anormales: desactivar una cuenta bancaria no borra el historial de pagos que ya la usaron — solo deja de ofrecerse como opción nueva.

Criterios de aceptación:

- Solo un usuario con rol `SUPER_ADMIN` puede crear, editar o desactivar cuentas bancarias.
- El cliente solo puede elegir cuentas activas (`is_active = true`) al pagar.

RF 8.7 — Programar, editar, cancelar y consultar historial de mantenimiento

Campo	Valor
Identificador	RF 8.7
Tipo	Requisito Funcional
Dominio	Fleet & Maintenance
¿Crítico?	No
Prioridad	Media

Entrada: el Administrador programa un mantenimiento futuro para un vehículo que no está **RETIRED**; puede iniciarlo, cancelarlo (en lugar de eliminarlo), o consultar el historial completo filtrando por vehículo y estado.

Salida: **VehicleMaintenance** creado como **SCHEDULED** (no cambia el estado del vehículo); al iniciarlo pasa a **IN_PROGRESS** (ver RF8.3); al cancelarlo pasa a **CANCELLED** y, si estaba **IN_PROGRESS**, el vehículo vuelve a **AVAILABLE**.

Descripción: el ciclo de vida de un mantenimiento tiene 4 estados: **SCHEDULED** → **IN_PROGRESS** → **COMPLETED**, con la posibilidad de pasar a **CANCELLED** desde **SCHEDULED** o **IN_PROGRESS**. Un mantenimiento nunca se elimina físicamente — "borrarlo" significa cancelarlo, igual que dar de baja un vehículo. Un mantenimiento **COMPLETED** o **CANCELLED** queda de solo lectura: ninguno de sus campos (tipo, fechas, costo, observaciones, foto) puede editarse después de cerrado.

Manejo de situaciones anormales: un mantenimiento solo puede iniciarse si el vehículo está **AVAILABLE**; no se puede programar mantenimiento para un vehículo **RETIRED**; un vehículo tiene como máximo un mantenimiento **IN_PROGRESS** a la vez, aunque puede tener varios **SCHEDULED**; dar de baja un vehículo cancela automáticamente sus mantenimientos **SCHEDULED** e **IN_PROGRESS**.

Criterios de aceptación:

- Un mantenimiento programado no afecta la disponibilidad del vehículo hasta que inicia.
- Un mantenimiento cerrado (**COMPLETED/CANCELLED**) no puede editarse.
- El historial permite filtrar por vehículo y por estado.

3.4 Requisitos No Funcionales

RNF 1 — Seguridad

El sistema requiere autenticación con control de roles (CLIENT/ADMIN/SUPER_ADMIN). Los datos personales se protegen según normativa aplicable. El acceso se controla mediante JWT: el access token expira a los 30 minutos y el refresh token a los 7 días, con rotación obligatoria en cada uso.

Criterios de aceptación: inicio de sesión requerido para operaciones protegidas · permisos correctamente aplicados por rol · ningún refresh token puede reutilizarse tras su rotación.

RNF 2 — Rendimiento

El sistema debe responder en menos de 2 segundos en el 95% de las operaciones, y actualizar la ubicación GPS de los vehículos en alquiler al menos cada 60 segundos.

Criterios de aceptación: $p95 < 2s$ · actualización de ubicación GPS cada $\leq 60s$.

RNF 3 — Usabilidad

La interfaz debe ser intuitiva, responsiva (funcionar en dispositivos móviles y de escritorio) y estar en español para el personal administrativo y los clientes.

Criterios de aceptación: funcionalidad equivalente en PC, tablet y móvil · interfaz de usuario en español, sin textos técnicos innecesarios.

RNF 4 — Mantenibilidad

La arquitectura debe ser modular (microservicios + hexagonal), con registro de errores accesible para el equipo técnico y documentación básica por servicio.

Criterios de aceptación: logs disponibles para el equipo técnico · cambios en un microservicio no deben requerir cambios en otros para funcionar (bajo acoplamiento).

RNF 5 — Integración

El sistema debe exponer contratos de API (OpenAPI) para cada microservicio, e integrar el dispositivo GPS y Cloudinary como almacenamiento de archivos (comprobantes de pago, códigos QR, fotos de perfil, de vehículos y de mantenimiento).

Criterios de aceptación: contratos OpenAPI documentados por servicio · fallas de un servicio externo (GPS, Cloudinary) no deben tumbar el resto del sistema.

RNF 6 — Disponibilidad

El sistema debe estar disponible al menos el 99% del tiempo mensual (una vez en producción), con recuperación ante fallos en menos de 10 minutos.

Criterios de aceptación: $uptime \geq 99\%$ · recuperación efectiva ante fallos ≤ 10 min.

RNF 7 — Consistencia de datos

Todos los microservicios comparten la misma base de datos SQL Server — cualquier migración de esquema debe coordinarse entre los servicios que leen/escriben las mismas tablas.

Criterios de aceptación: ninguna migración de esquema se despliega sin revisión de los servicios que dependen de esas tablas · las migraciones se versionan con Liquibase.